1

```
# Databricks notebook source
from pyspark.sql.functions import current_timestamp
def add_ingestion_date(input_df):
  output_df = input_df.withColumn("ingestion_date", current_timestamp())
  return output_df
```

2

```
def re_arrange_partition_column(input_df, partition_column):
  column_list = []
  for column_name in input_df.schema.names:
    if column_name != partition_column:
      column_list.append(column_name)
  column_list.append(partition_column)
  output_df = input_df.select(column_list)
  return output_df
```

3

```
def overwrite_partition(input_df, db_name, table_name, partition_column):
  output_df = re_arrange_partition_column(input_df, partition_column)
  spark.conf.set("spark.sql.sources.partitionOverwriteMode","dynamic")
  if (spark._jsparkSession.catalog().tableExists(f"{db_name}.{table_name}")):
    output_df.write.mode("overwrite").insertInto(f"{db_name}.{table_name}")
  else:
    output_df.write.mode("overwrite").partitionBy(partition_column).format("parquet").saveAsTable(f"{db_name}.{table_name}")
```

4

```
def df_column_to_list(input_df, column_name):
  df_row_list = input_df.select(column_name) \
                        .distinct() \
                        .collect()

  column_value_list = [row[column_name] for row in df_row_list]
  return column_value_list
```

5

```
def merge_delta_data(input_df, db_name, table_name, folder_path, merge_condition, partition_column):
  spark.conf.set("spark.databricks.optimizer.dynamicPartitionPruning","true")

  from delta.tables import DeltaTable
  if (spark._jsparkSession.catalog().tableExists(f"{db_name}.{table_name}")):
    deltaTable = DeltaTable.forPath(spark, f"{folder_path}/{table_name}")
    deltaTable.alias("tgt").merge(
        input_df.alias("src"),
        merge_condition) \
      .whenMatchedUpdateAll()\
      .whenNotMatchedInsertAll()\
      .execute()
  else:
    input_df.write.mode("overwrite").partitionBy(partition_column).format("delta").saveAsTable(f"{db_name}.{table_name}")
```

6